

Megapot V2: Supplementary Audit

Joint audit report

[montecristo](#) | [hgrano](#) | [0xnev](#)

March 6, 2026

Contents

Summary	4
Severity Classifications	4
Scope	5
Low Severity	5
L-01 Batch execution time is incorrectly estimated	5
Target	5
Finding Description and Impact	5
Recommended Mitigation Steps	6
Resolution	6
L-02 Auto compounding via batch purchase facilitator can be reverted	6
Target	6
Finding Description and Impact	6
Recommended Mitigation Steps	7
Resolution	7
Informational	7
I-01 Unnecessary ownership check	7
Target	7
Finding Description and Impact	7
Recommended Mitigation Steps	7
Resolution	8
I-02 Unnecessary drawing ID validation	8
Target	8
Finding Description and Impact	8
Recommended Mitigation Steps	8
Resolution	8

I-03	Unnecessary check on drawing tier payouts	8
	Target	8
	Finding Description and Impact	8
	Recommended Mitigation Steps	9
	Resolution	9
I-04	Redundant ticket NFT tracking for the vault	9
	Target	9
	Finding Description and Impact	9
	Recommended Mitigation Steps	9
	Resolution	10
I-05	Redundant jackpotLock state check	10
	Target	10
	Finding Description and Impact	10
	Recommended Mitigation Steps	10
	Resolution	10
Disclaimer		10

Scope

The coordinationlabs/megaplot-v2-contracts repository at commit [138de89](#) was audited with the following files being in scope:

```
contracts
├── TicketAutoCompoundVault.sol
```

Additionally, the same repository was audited at commit [84580dd](#) with the below scope:

```
contracts
├── Jackpot.sol
├── JackpotLPManager.sol
├── interfaces
│   └── IJackpotLPManager.sol
```

All findings for both scopes are collated into this single report.

Low Severity

L-01 Batch execution time is incorrectly estimated

Target

[TicketAutoCompoundVault.sol:275-278](#)

Finding Description and Impact

When compound tickets are purchased, the batch execution time is estimated. If the estimated time cannot meet the current drawing deadline, the compound request may be rejected:

```
@>      uint256 executionTimeSeconds = Math.ceilDiv(ticketsToBuy,
ticketPurchaseRate) * 60;
      if (block.timestamp + executionTimeSeconds >= _drawingState.drawingTime)
      {
          revert InsufficientTimeForBatchExecution();
      }
```

In L275, the execution time is estimated as the following (ignoring rounding and minute-scaling):

```
ticketsToBuy / ticketPurchaseRate
```

The problem with this approach is that it implicitly assumes `BatchPurchaseFacilitator` has no other batches to process. Instead, the correct estimation should be:

```
(existingTicketsToProcess + ticketsToBuy) / ticketPurchaseRate
```

However, `BatchPurchaseFacilitator` doesn't expose a method to track `existingTicketsToProcess`. This is completely handled off-chain.

As a result, the compound vault may accept a compound request which cannot be processed until the deadline. The registered batch will then be cancelled due to the wrong drawing and all the prize will be refunded to the user.

Recommended Mitigation Steps

The fix isn't trivial. The estimation logic could be removed and replaced with a hardcoded deadline for compounding (e.g. 1 hour before the next drawing), since batch execution time estimation may not be possible on chain.

Resolution

Megapot: Fixed by [b3c9b70b9a973fddd2d22fe2c4c30aa48c5de8b3](#).

Audit team: Verified.

L-02 Auto compounding via batch purchase facilitator can be reverted

Target

[TicketAutoCompoundVault.sol](#)

Finding Description and Impact

In the `_purchaseTicketsAndUpdateState` internal function that performs the ticket purchase utilising the claimed winnings, there is an option to perform a purchase via the `batchFacilitator` or the `jackpot` contract itself. Notice that there is a check to ensure the current user does not have an active order before a batch order creation, i.e. the existing order must have been executed and/or cancelled by the user:

```
if (ticketsToBuy > 0) {
    if (useBatchPurchase) {
        // Large order - verify sufficient time for batch execution before
        // drawing
        // executionTime (in seconds) = (ticketsToBuy / ticketPurchaseRate) * 60
        uint256 executionTimeSeconds = Math.ceilDiv(ticketsToBuy,
            ticketPurchaseRate) * 60;
        if (block.timestamp + executionTimeSeconds >= _drawingState.drawingTime)
        {
            revert InsufficientTimeForBatchExecution();
        }
        // Check for active batch order
    @> if (batchFacilitator.hasActiveBatchOrder(msg.sender)) revert
        ActiveBatchOrderExists();
        usdc.forceApprove(address(batchFacilitator), usdcSpent);
        batchFacilitator.createBatchOrder(
            msg.sender,
            uint64(ticketsToBuy),
            new IJackpot.Ticket[](0),
            _referrers,
            _referralSplit
        );
    }
```

Since `createBatchOrder` is permissionless, any user can cause the mentioned check to be invoked by simply allocating a `minimumTicketCount` (e.g. 1) to the recipient to cause a non-zero `remainingUSDC` to be assigned to the user:

```
function hasActiveBatchOrder(address _recipient) external view returns (bool) {
```

```
    return batchOrders[_recipient].remainingUSDC > 0;
}
```

If the DoS is sustained over long periods of time, the potential impact would be the associated consequences of the purchase of tickets (winnings, referral fees and protocol fees).

Recommended Mitigation Steps

A specific user can consider batch calling a `cancelBatchOrder` and `depositAndCompound` call to prevent such a DoS. Alternatively, consider calling `executeBatchOrder` for the user before performing a compounding deposit.

Resolution

Megapot: Acknowledged. [Response by Megapot team:](#)

I think we're generally ok with the griefing potential. We'll have infra that is churning through these orders leaving small windows to execute on it. There are some other mitigations we can take off-chain as well (like canceling before calling as you mentioned). We definitely optimized for simplicity here - so if we see some griefing activity we may end up re-evaluating.

Informational

I-01 Unnecessary ownership check

Target

[TicketAutoCompoundVault.sol:237](#)

Finding Description and Impact

The following check on ownership of the ticket NFT is redundant:

```
function _validateAndTransferTickets(uint256[] calldata _ticketIds) internal {
    uint256[] memory tierIds = jackpot.getTicketTierIds(_ticketIds);
    uint256 currentDrawingId = jackpot.currentDrawingId();
    for (uint256 i; i < _ticketIds.length; ++i) {
        // Verify ownership (also catches duplicate ticketIds since prior
        // iteration transfers to vault)
        @> if (IERC721(address(jackpotNFT)).ownerOf(_ticketIds[i]) != msg.sender)
        revert TicketNotOwnedByUser();
    }
}
```

This is because the ownership is subsequently checked during `JackpotTicketNFT::transferFrom` anyway.

Recommended Mitigation Steps

Remove the check on NFT ownership to avoid an unnecessary call and storage reads.

Resolution

Megapot: Fixed by [e0bf2c144e6fb5c78b5a5c75923b39952538e2e7](#).

Audit team: Verified.

I-02 Unnecessary drawing ID validation

Target

[TicketAutoCompoundVault.sol:241](#)

Finding Description and Impact

The validation of drawing ID shown below is not necessary because this is already checked within `Jackpot::claimWinnings` (line 428):

```
function _validateAndTransferTickets(uint256[] calldata _ticketIds) internal {
    uint256[] memory tierIds = jackpot.getTicketTierIds(_ticketIds);
    uint256 currentDrawingId = jackpot.currentDrawingId();
    for (uint256 i; i < _ticketIds.length; ++i) {
        // [...]
        // Verify drawing completed
        IJackpotTicketNFT.TrackedTicket memory ticket = jackpotNFT.getTicketInfo
            (_ticketIds[i]);
    }
    if (ticket.drawingId >= currentDrawingId) revert DrawingNotCompleted();
}
```

Recommended Mitigation Steps

Remove the check on drawing ID here to avoid an unnecessary call and storage reads.

Resolution

Megapot: Fixed by [d6329307b082c92ca16146fc20b94351463048e5](#).

Audit team: Verified.

I-03 Unnecessary check on drawing tier payouts

Target

[TicketAutoCompoundVault.sol:244-245](#)

Finding Description and Impact

Checking if the ticket is a winner or not is technically not necessary here because a before/after balance check is already done in `TicketAutoCompoundVault::_claimWinnings`:

```
function _validateAndTransferTickets(uint256[] calldata _ticketIds) internal {
    uint256[] memory tierIds = jackpot.getTicketTierIds(_ticketIds);
    uint256 currentDrawingId = jackpot.currentDrawingId();
    for (uint256 i; i < _ticketIds.length; ++i) {
        // [...]
    }
}
```



```
// Verify ticket is a winner
uint256[12] memory tierPayouts = jackpot.getDrawingTierPayouts(ticket.
    drawingId);
@> if (tierPayouts[tierIds[i]] == 0) revert TicketNotWinner();
```

Recommended Mitigation Steps

Remove the check on `tierPayouts` here to avoid an unnecessary call and storage reads. Instead, an assertion could be added to `TicketAutoCompoundVault::_claimWinnings` to check for non-zero balance increase from winnings.

Resolution

Megaplot: Fixed by [d6329307b082c92ca16146fc20b94351463048e5](#).

Audit team: Verified.

I-04 Redundant ticket NFT tracking for the vault

Target

[TicketAutoCompoundVault.sol:253](#), which calls: [JackpotTicketNFT.sol:223-234](#)

[TicketAutoCompoundVault.sol:247](#), which calls: [JackpotTicketNFT.sol:239-246](#)

Finding Description and Impact

The `JackpotTicketNFT` contract keeps track of all ticket IDs owned by each address within its `_beforeTokenTransfer` and `_afterTokenTransfer` functions. The issue is that the vault is always a recipient of the NFTs from the user and then immediately the NFTs will be burned in the same transaction (when winnings are claimed). Therefore, at the start of each transaction the `TicketAutoCompoundVault` has no data stored for it within the `userTickets` mapping on the `JackpotTicketNFT`. Then, during the transaction, a write is done to this (initially zero) storage within the `_afterTokenTransfer` (during the transfer from the user to the contract). Finally, during the NFT burn the storage will get cleared back to zero (although a gas refund is earned, it will be capped at 1/5 of the total gas usage). This becomes quite gas-heavy for a large number of tickets. Moreover, the ticket tracking for the `TicketAutoCompoundVault` is redundant as its ticket ownership never persists after the end of the transaction.

Recommended Mitigation Steps

Possible solutions include:

- Disabling tracking for specific addresses such as the vault (using an `if` statement in the `_beforeTokenTransfer` and `_afterTokenTransfer` functions).
- Use events instead of on-chain storage to track NFTs owned by each address (if on-chain lookup is not required).
- Change the `Jackpot` to permit allowed addresses to call `claimWinnings` (instead of just NFT owners), thereby avoiding the transfer step from user to vault.

Resolution

Megapot: Acknowledged.

I-05 Redundant jackpotLock state check

Target

[TicketAutoCompoundVault.sol](#)

Finding Description and Impact

The `depositAndCompound` function performs an initial `jackpotLock` state check to ensure ticket purchases are not performed while the current draw is not completed:

```
IJackpot.DrawingState memory drawingState = jackpot.getDrawingState(jackpot.  
    currentDrawingId());  
@> if (drawingState.jackpotLock) revert JackpotErrors.JackpotLocked();
```

This is actually not required, given the underlying `jackpot` and `batchFacilitator` contracts check this state via `_validateBuyTicketInputs` and `createBatchOrder` respectively.

Recommended Mitigation Steps

Consider removing this check. Alternatively, it could be shifted into the `_validateAndTransferTickets` function for better code consistency. Note that it could be good to maintain this check since `batchFacilitator` can be switched out in the future which may omit the jackpot lock status check.

Resolution

Megapot: Acknowledged.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. It is a process limited by available time, resources, and expertise, aimed at identifying as many issues as possible. We cannot assure complete security after the review, nor can we promise that all issues will necessarily be discovered. To improve overall security, additional reviews, bug bounty initiatives, and continuous on-chain monitoring are highly recommended.